

Logan: A Distributed Online Log Parser

Amey Agrawal, Rohit Karlupia, Rajat Gupta

Qubole India Pvt. Ltd.

Bengaluru, India

{ameya, rohitk, rgupta}@qubole.com

Abstract—Logs serve as a critical tool for debugging and monitoring applications. However, gaining insights from unstructured logs is difficult. Hence, many log management and analysis applications first parse logs into structured templates. In this paper, we train a data-driven log parser on our new Apache Spark dataset, the largest application log dataset yet. We implement a distributed online algorithm to accommodate for the large volume of data. We also devise a new metric for evaluation of parsers when labeled data is unavailable. We show that our method generalizes over diverse datasets without any parameter tuning or domain-specific inputs from the user. When evaluated on publicly available HDFS dataset our method performs 13x faster than the previous state-of-the-art.

Index Terms—Log parsing, Online algorithm, Distributed processing

I. INTRODUCTION

In recent times with the advent of cloud computing, large-scale distributed computing has become more accessible and affordable. Applications scaling up to thousands of nodes can generate massive log files within minutes. Obtaining insights from these logs requires parsing the unstructured data. Considering the ever-increasing number of third-party software libraries, data-driven methods provide a scalable solution to the parsing problem.

In this paper, we introduce Logan, a distributed log parsing technique designed for large-scale datasets. Each log event can be considered as a string generated from a template where some tokens are constants while others represent values of certain variables. Log parsing techniques try to identify unique templates which correspond to specific system events.

Log parsing methods can be broadly divided into system-specific knowledge based algorithms and data mining based approaches. The former involves using either regular expressions or source code analysis. Although several data mining based methods have been developed in the past, they use small datasets and manually identified regular expressions as ground truth for evaluation. We introduce two new datasets comprised of Presto [1] and Apache Spark [2] application logs as a tool to develop more robust and scalable algorithms. We also introduce a new evaluation metric for log parsing methods which does not require manually labeled ground truth, facilitating evaluation against larger datasets.

To maintain efficacy while processing large amount of data, we employ a Longest Common Subsequence (LCS) based method which can be distributed to scale with the data while processing in a quasi-streaming fashion to avoid memory bottlenecks. We make a few key observations based

on the nature of system generated logs, which enable us to develop an application agnostic pre-processing pipeline that improves the robustness of our method compared to previous methods. Streaming algorithms for log parsing [3], [4] perform a template update step where unmatched tokens between the incoming line and the matched event template are replaced by wildcards (“*”). In the proposed approach new constraints have been set to limit the information loss during the merge step as the number of log lines increase.

Our modifications markedly improve log parsing performance over existing methods. On the HDFS dataset, we reduce running time from 379 seconds [5] to 28 seconds. For our newly introduced Apache Spark log dataset, we process 96 million records in 409 seconds compared to 896 seconds for 13 million records of the previous best.

II. BACKGROUND

Data-driven log parsing has been widely studied over the years. Vaarandi et al. [6] identify the most frequently occurring tokens and use them as features to represent each template. Another group of techniques LKE [7], POP [8], IPLoM [5] cluster all lines into coarse partitions which are then iteratively divided in an attempt to create clusters where all the lines correspond to a single template. These initial groups are created by finding edit distance between each pair of lines in LKE [7], which makes this process costly both in terms of memory and compute utilization. POP [8] and IPLoM [5] create the initial clusters by the number of tokens in the line, they further recognize the tokens which appear most frequently at a given position and split the cluster on these token positions. POP further recursively divides these clusters by relative frequency analysis, while IPLoM find bijective relations between unique tokens for further subdivision.

More relevant to our work are [3], [4] and [9] which perform a streaming analysis of the log lines, enabling the processing of large datasets by reducing memory bottleneck. Online nature of these methods also allows introducing templates from new lines without retraining on the complete data. Du et al. [3] employ LCS for matching templates similar to our method. They attain faster matching using prefix trees. Drain uses a technique similar to [4], [8] and [5]. It first divides the incoming lines into groups based on length. Each group further contains a set of unique templates. For each incoming line a token equality based similarity score is computed against each template. The templates are merged if the similarity score is

greater than a threshold. This threshold is increased with the number of wildcards in a template.

POP [8] uses Apache Spark to perform distributed training similar to us. They observe that the non-distributed version of their algorithm runs faster than its distributed counterpart due to communication overheads. The distributed version becomes more efficient when the datasets become larger. Our method on the other hand scales even for relatively smaller datasets allowing us to outperform the existing methods by a large factor.

III. DISTRIBUTED LOG PARSING

In this section, we describe three key stages of our algorithm, namely template extraction from individual shards, hard merge of templates collected from each shard and the final soft merge.

A. Architecture

We use Apache Spark running on YARN [10] during the training stage. The Spark driver lists input log files that are stored on HDFS [11]. To avoid task skew [12], input files are sharded before further processing. The driver assigns each input shard to a separate executor task, which extracts the templates. *Hard merge* concatenates templates from all the tasks to form a single list, while removing identical templates which might have been discovered independently in different tasks. This operation is executed as a `treeReduce` operation. The output is collected on the spark driver, which further performs *soft merge* as described in subsection III-I to produce the final list of templates.

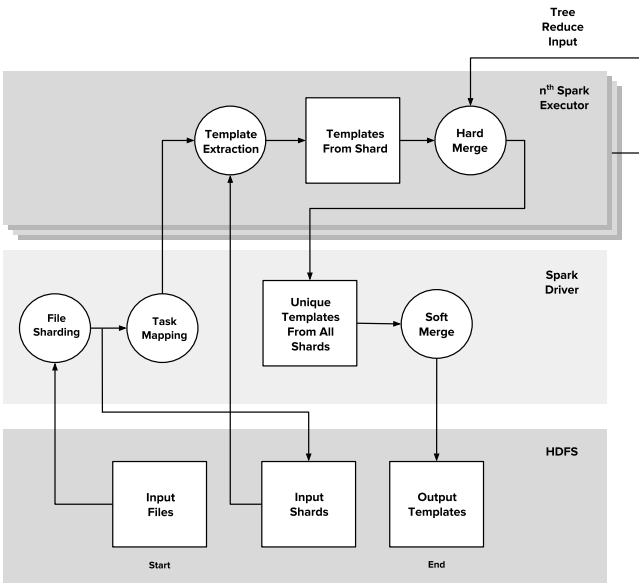


Fig. 1: Overview of Logon Training

B. Template Extraction Overview

In the template extraction phase, we first preprocess each line to identify obvious variables. The preprocessed line is then matched against the existing templates to identify the best match. In case we do not find any suitable match we create a new template from the given line. Since all the lines match to a very small set of templates, we maintain a hashmap of preprocessed lines and the corresponding template as a cache. We observe that we hit the cache more than 98% percent times across all the benchmark tasks, providing a substantial speedup.

C. Preprocessing of Log Lines

Given an input line, we split it on spaces to create a sequence of tokens. We analyze these tokens to identify URLs and file paths using regular expressions. These URLs are then replaced with a wildcard. Following which we again split each token on every non-alphanumeric character and replace all the decimal and hexadecimal numbers from the resulting tokens. This enables us to handle dates (“31/01/1997”), IP addresses (“127.0.0.1”) and other similar tokens without using specific regular expressions. We further replace all the tokens that contain at least one numeric character and less than three alphabetical characters. This preserves tokens like “HTTP505” while discarding tokens like “M20”. We observe that most meaningful tokens are shorter than 50 characters, hence, we remove all tokens longer than 50 characters.

TABLE I: LCS matching with tokens split on spaces

<i>Id</i>	<i>Sequence</i>	<i>Tem-plate Id</i>	<i>LCS length with s_1</i>
s_1	App > 18/08/29 11:09:14 main INFO SparkContext: Added JAR hdfs:///example-1.jar at spark:///127.0.0.1:9090/example-1.jar with timestamp 1538642318	t_1	-
s_2	App > 19/08/29 12:10:25 main INFO SparkContext: Added JAR hdfs:///example-2.jar at spark:///127.0.0.1:9090/example-2.jar with timestamp 1538642450	t_1	10
s_3	App > 19/08/29 13:24:51 main INFO SparkContext: Updated JAR hdfs:///example-1.jar at spark:///127.0.0.1:9090/example-1.jar with timestamp 1538642872	t_2	11

D. Prefiltering of templates

The prefiltering step eliminates trivial cases to reduce the computation in the following stages. He et al. [4] employ a similar strategy where they narrow down the search by only comparing with templates of identical token length as the input sequence. Though effective, this does not accommodate for templates which could produce variable length log lines. A more lenient approach is adopted by [3], where they match

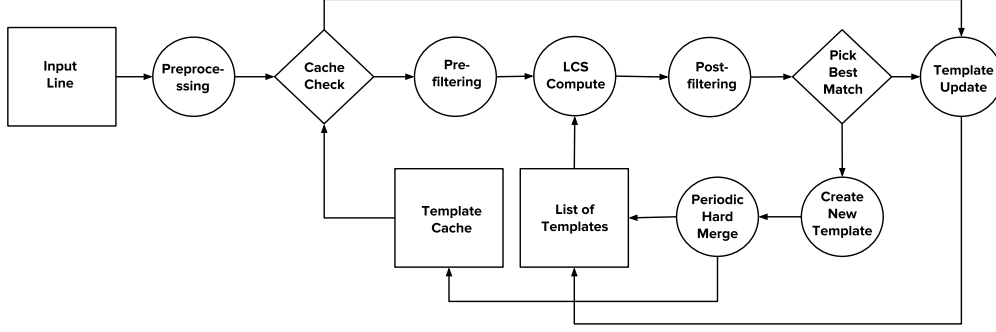


Fig. 2: Overview of Logan Training

TABLE II: LCS matching with Logan’s preprocessing

<i>Id</i>	<i>Sequence</i>	<i>Tem- plate Id</i>	<i>LCS length with s_1</i>
s_1	App * main INFO SparkContext: Added JAR * at * with timestamp *	t_1	-
s_2	App * main INFO SparkContext: Added JAR * at * with timestamp *	t_1	9
s_3	App * main INFO SparkContext: Updated JAR * at * with timestamp *	t_2	8

all templates with length greater than half and less than twice the input sequence length. We define a *matching tolerance* $m(t)$ for every template t in the set of templates T , which we describe in more detail in subsection III-G. We select template t for matching with input sequence s such that it satisfies,

$$|t.\text{length} - s.\text{length}| < m(t) \quad (1)$$

E. LCS for Sequence Matching

Longest Common Subsequence (LCS) is a well studied problem and is used for multiple sequence matching applications including DNA matching [13] and version control systems [14]. LCS is also adopted as template similarity metric in [3]. Notably, we identify the effect of preprocessing on performance of LCS based pattern matching. Consider the triplet represented in Table I, two of which are generated from template t_1 while the third belongs to template t_2 . Splitting tokens only based on spaces as used by [3] leads to a false match between sequence s_1 and s_3 . By removing obvious variables, our preprocessing enables us to identify the right match as shown in Table II.

F. Postfiltering and Template Update

The postfiltering stage enforces constraints on sequence similarity to avoid incorrectly matching the input sequence to an existing template when no real match exists. When we match an input sequence to an existing sequence we replace the conflicting tokens with the wildcard, consequently

incorrect matching leads to loss of informative tokens as illustrated in Fig. 3.

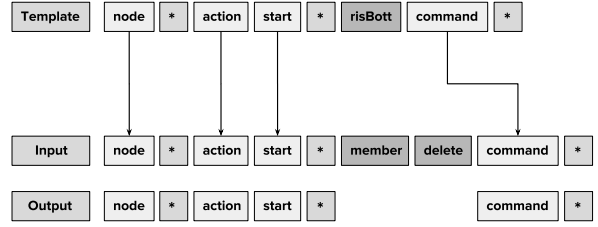


Fig. 3: Merging two sequences with different templates.

We also observe that this effect can compound in certain cases. For example, a template t , instantiated with a token length of 20, undergoes a merge that reduces its length to 18, further the updated template is again merged resulting in a template length of 16 and so on. We design the constraints to discourage larger changes in the template. Since most variables are identified in the preprocessing stage itself, we can set a very low *matching tolerance*.

For a template t^i which has been transformed $i - 1$ times in its lifetime and input sequence s we define the post-filtering constraints as,

$$\text{realLength}(s) - \text{LCS}(t^i, s) \geq m(t^i) \quad (2)$$

$$\text{realLength}(t^i) - \text{LCS}(t^i, s) \geq m(t^i) \quad (3)$$

$$\text{realLength}(t^{i-1}) - \text{LCS}(t^i, s) \geq m(t^i) \quad (4)$$

Where realLength is a function that returns the number of non-wildcard tokens in the input sequence.

G. Configuring Matching Tolerance

We design the *matching tolerance* function to reduce the tolerance for sequence dissimilarity as the fraction of wildcards in a given template goes up. As the sequence gets smaller preserving each informative token gets more critical.

We empirically choose values for the following function, which performs well on a wide variety of tasks.

$$m(t) = \begin{cases} 0, & \text{if } t.\text{length} \leq 5, \text{numWildcards}(t) \geq 2 \\ 1, & \text{if } t.\text{length} \leq 5, \text{numWildcards}(t) < 2 \\ 1, & \text{if } t.\text{length} > 5, \\ & \text{numWildcards}(t) \geq 0.3 * t.\text{length} \\ 2, & \text{if } t.\text{length} > 5, \\ & \text{numWildcards}(t) < 0.3 * t.\text{length} \end{cases} \quad (5)$$

H. Periodic Hard Merge

Templates t_n and t_m might be initialized as different templates, but after few transformations as all the variables are identified they might converge to identical sequence such that $t_n^i = t_m^j$. Since time complexity of the LCS compute step is linear in the number of templates, we periodically perform *hard merge* over the list of templates to remove any duplicates.

I. Soft merge

Once we collect all the templates identified by each executor task and combine them via *hard merge*, there might still be some variables which have not been identified. If a certain variable does not change within a shard, we would fail to identify such a variable, hence, we perform LCS based matching within the list of templates to merge the similar templates. This also helps in matching performance as the number of templates are reduced.

IV. APPLICATION IN SOFTWARE DEBUGGING

A. Overview

Identifying the root cause of failure for applications with large logs is challenging. Hence, spotting the specific lines which might indicate the reason for failure promises to save both time and effort for developers. We recognize the templates that frequently appear in certain application logs by parsing logs of successful jobs. As we remove the lines matching these templates from logs for failed jobs, we are left with error logs, stack traces and other anomalies. We also create a feedback loop to continuously improve the knowledge base of templates.

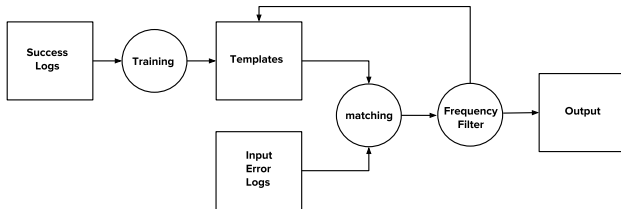


Fig. 4: Overview of Anomaly Detection with Logan

B. Matching

Sometimes in a successful job, certain instructions might fail initially but succeed on retry, hence if we use each template learned from successful job logs for filtering we might miss some important lines. On our Spark application log dataset, we observe that top twenty percent templates match ninety-eight percent lines. Hence, we use a greedy policy to select the minimum number of templates required to achieve desired coverage on the training set. We further note that some templates such as those for job start and termination notification only appear once in each job but are present in every job. Therefore, we also select the templates that appear in more than eighty percent of jobs. We employ more lenient constraints for pre-filtering and post-filtering.

Pre-filtering constraints:

$$t.\text{length} - \frac{1}{\alpha} * s.\text{length} > 0 \quad (6)$$

$$\alpha * s.\text{length} - t.\text{length} > 0 \quad (7)$$

Post-filtering constraints:

$$\text{LCS}(t, s) - \frac{1}{\beta} * \text{realLength}(s) > 0 \quad (8)$$

$$\text{LCS}(t, s) - \frac{1}{\beta} * \text{realLength}(t) > 0 \quad (9)$$

Where α and β are empirically set to 2 and 1.5 respectively.

C. Frequency Filter and Feedback Loop

The logs we used for training only represent a small subset of log templates that could exist in a given application. When detecting anomalous logs in a job which contains new templates, we might unnecessarily output lines which do not help in debugging the application. To avoid this, we keep track of the number of times a template appears in a given file. If the number exceeds the set threshold we remove the template from the output and update the list of templates. This threshold value must be set depending upon the application type. We identify two hundred and fifty as a suitable value for Spark applications.

TABLE III: Summary of Datasets

Dataset	Number of Lines	Total Size
Zookeeper	74,380	10 MB
HPC	433,490	32 MB
BGL	4,747,963	109 MB
Presto-5.2M	5,200,000	1486 MB
HDFS	11,175,629	1577 MB
Spark-13M	13,000,000	2104 MB
Presto	16,222,793	4734 MB
Spark	96,503,051	14758 MB

TABLE IV: Loss measurement for Logan and other methods ($\theta = 1.5$)

	<i>Zookeeper</i>	<i>HPC</i>	<i>BGL</i>	<i>Presto-5.2M</i>	<i>HDFS</i>	<i>Spark-13M</i>	<i>Presto</i>	<i>Spark</i>
SHISO [9]	9.964	14.178	-	-	-	-	-	-
Spell [3]	11.598	12.690	-	-	-	-	-	-
Drain [4]	9.513	11.248	18.970	140.600	7.618	18.159	-	-
IPLoM [5]	9.585	12.317	22.318	30.924	7.156	24.100	-	-
Logan (Single Node)	9.920	10.999	13.661	17.725	7.628	17.725	-	-
Logan (Distributed)	-	-	13.085	-	7.688	-	25.293	22.629

TABLE V: Number of templates identified by Logan and other methods

	<i>Zookeeper</i>	<i>HPC</i>	<i>BGL</i>	<i>Presto-5.2M</i>	<i>HDFS</i>	<i>Spark-13M</i>	<i>Presto</i>	<i>Spark</i>
SHISO [9]	42	90	-	-	-	-	-	-
Spell [3]	168	230	-	-	-	-	-	-
Drain [4]	89	147	1202	1466	48	1001	-	-
IPLoM [5]	91	119	2765	3748	41	738	-	-
Logan (Single Node)	100	138	301	2391	48	895	-	-
Logan (Distributed)	-	-	258	-	49	-	5514	2979

V. EXPERIMENTS

A. Experimental Setup

We perform a comparative analysis with IPLoM [5], SHISO [9], POP [8], Spell [3] and Drain[4]. All our experiments were performed on Amazon EC2 *r3.8xlarge* instances with 32 virtual cores. We use an Apache Spark cluster with five *r3.8xlarge* instances for Logan in distributed mode. We perform evaluation on four public [15], [16], [17], [4] and two new datasets. Table III provides a description of each of these datasets. The Apache Spark and Presto datasets that we introduce in this work are the largest log datasets to be studied in terms of the number of lines and the number of templates respectively. For these two datasets, we also introduce a smaller counterpart for easier comparison with non-distributed techniques. We omit the results if a method takes too long to finish a particular task.

B. Evaluation Metric

The effectiveness of log parsing algorithm is evaluated using accuracy and F-measure in the past works. However, this requires manually defining regular expressions for each template in the dataset. To avoid this in large datasets, we propose a new evaluation metric.

$$\text{avgTokensLost}(t) = \text{avgMatchLength}(t) - t.\text{length} \quad (10)$$

$$\text{qualityFactor}(T) = \frac{1}{T.\text{length}} \sum_{i=0}^{T.\text{length}} \left(\frac{\text{avgTokensLost}(t_i)}{t_i.\text{length}} \right)^2 \quad (11)$$

$$\text{lengthFactor}(T) = (\log T.\text{length})^\theta \quad (12)$$

$$\text{loss}(T) = \text{qualityFactor} + \text{lengthFactor} \quad (13)$$

Where, $\text{avgMatchLength}(t)$ is the average length of all sequences matched to template t during training.

The *qualityFactor* penalizes over generalized templates; where possibly informative tokens are converted into wild-cards. On the other hand, *lengthFactor* adds a penalty for the inclusion of too many templates which might be an indication of inability to identify variables. The value of θ can be adjusted to control the importance of the two factors depending upon the use-case.

C. Observations

1) *Parse Quality*: As depicted in Tables IV and V, SHISO identifies fewer templates than all other methods. The higher loss along with fewer templates suggests that important tokens are lost. On the other hand, Spell fails to identify some of the obvious variables due to its simple preprocessing and creates too many templates. IPLoM performs well across most datasets, however, it fails to identify a few variables in certain templates in the BGL dataset. Drain's loss on Presto dataset is much higher than the other algorithms due to over-parsing. Logan shows a consistent behavior across datasets and obtains the lowest loss on six out of the eight tasks while requiring zero parameter tuning.

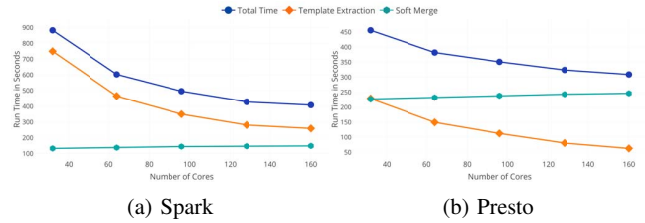


Fig. 5: Running time vs. Number of cores

TABLE VI: Run time performance of Logan and other methods (in seconds)

	<i>Zookeeper</i>	<i>HPC</i>	<i>BGL</i>	<i>Presto-5.2M</i>	<i>HDFS</i>	<i>Spark-13M</i>	<i>Presto</i>	<i>Spark</i>
SHISO [9]	129.698	656.905	-	-	-	-	-	-
Spell [3]	13.602	76.696	-	-	-	-	-	-
Drain [4]	3.481	16.269	156.177	436.075	559.679	7044.181	-	-
IPLoM [5]	2.188	11.258	93.231	583.550	379.038	896.225	-	-
Logan (Single Node)	1.132	3.782	24.169	68.347	53.390	119.032	-	-
Logan (Distributed)	-	-	23.035	-	28.767	-	309.090	409.632

2) *Run Time*: Logan outperforms all the previous methods in both non-distributed and distributed settings alike as shown in Table VI. However, as Fig. 5a and 5b show, our method scales in a sub-linear fashion which can be attributed to the hard and soft merge stages. The number of hard merge operations increase with the number of shards, but since this operation is parallelized the relative overhead is low. In contrast, the soft merge process run can't be made parallel and introduces a bottleneck. The time required for soft merge stage depends on the number of unique templates and not on the size of the dataset. Hence, this bottleneck is more prominent for Presto dataset, which contains 5514 templates as opposed to 2979 templates in Spark dataset.

VI. CONCLUSION

We have demonstrated the effectiveness of the presented distributed online log parsing system on various datasets setting a new level of performance. We have also introduced two new datasets which pose new challenges in log parsing due to their diversity of templates and size. In addition, an analysis was presented based on a new evaluation metric, allowing us to gain insights about parse quality on datasets where ground truth templates are unavailable. Our experiments confirm that the improved preprocessing and updated constraints allow us to generalize on multiple datasets without losing parse quality.

REFERENCES

- [1] "Presto: Distributed sql query engine for big data." [Online]. Available: <https://prestodb.io/>
- [2] "Apache spark - unified analytics engine for big data." [Online]. Available: <http://spark.apache.org/>
- [3] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *Data Mining (ICDM), 2016 IEEE 16th International Conference on*. IEEE, 2016, pp. 859–864.
- [4] P. He, J. Zhu, P. Xu, Z. Zheng, and M. R. Lyu, "A directed acyclic graph approach to online log parsing," *arXiv preprint arXiv:1806.04356*, 2018.
- [5] A. Makanju, A. N. Zincir-Heywood, and E. E. Milios, "A lightweight algorithm for message type extraction in system application logs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 24, no. 11, pp. 1921–1936, 2012.
- [6] R. Vaarandi, "A data clustering algorithm for mining patterns from event logs," in *IP Operations & Management, 2003.(IPOM 2003). 3rd IEEE Workshop on*. IEEE, 2003, pp. 119–126.
- [7] Q. Fu, J.-G. Lou, Y. Wang, and J. Li, "Execution anomaly detection in distributed systems through unstructured log analysis," in *Data Mining, 2009. ICDM'09. Ninth IEEE International Conference on*. IEEE, 2009, pp. 149–158.
- [8] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Transactions on Dependable and Secure Computing*, 2017.
- [9] M. Mizutani, "Incremental mining of system log format," in *Services Computing (SCC), 2013 IEEE International Conference on*. IEEE, 2013, pp. 595–602.
- [10] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth *et al.*, "Apache hadoop yarn: Yet another resource negotiator," in *Proceedings of the 4th annual Symposium on Cloud Computing*. ACM, 2013, p. 5.
- [11] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system," in *Mass storage systems and technologies (MSST), 2010 IEEE 26th symposium on*. Ieee, 2010, pp. 1–10.
- [12] M. Armbrust, T. Das, A. Davidson, A. Ghodsi, A. Or, J. Rosen, I. Stoica, P. Wendell, R. Xin, and M. Zaharia, "Scaling spark in the real world: performance and usability," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1840–1843, 2015.
- [13] S. J. Shyu and C.-Y. Tsai, "Finding the longest common subsequence for multiple biological sequences by ant colony optimization," *Computers & Operations Research*, vol. 36, no. 1, pp. 73–91, 2009.
- [14] L. Torvalds and J. Hamano, "Git: Fast version control system," URL <http://git-scm.com>, 2010.
- [15] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. ACM, 2009, pp. 117–132.
- [16] L. LANS, "Operational data to support and enable computer science research."
- [17] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Dependable Systems and Networks, 2007. DSN'07. 37th Annual IEEE/IFIP International Conference on*. IEEE, 2007, pp. 575–584.